

Лабораторная работа №1 «Алгоритм обедающих философов»

Введение

Взаимное исключение — требование, согласно которому во время выполнения критической области одного процесса ни один другой процесс не должен выполняться в этой же критической области. Различная скорость выполнения процессов и их число, произвольные задержки передачи сообщений и отсутствие полной информации о состоянии всей системы и общей памяти в распределенных системах делают реализацию взаимного исключения одной из фундаментальных проблем в области распределенных вычислений.

Исходные данные

Следует использовать исходные данные из подготовительной работы.

Задание

Данная работа выполняется на основе распределенной системы, реализованной в подготовительной работе. При этом понятие «полезной» работы дочернего процесса определяется следующим образом: каждый дочерний процесс должен $N = process_local_id * 5$ раз распечатать сообщение, определенное строкой форматирования `log_loop_operation_fmt`, посредством вызова функции `print()`, входящей в состав прилагаемой библиотеки `libruntime.so`. Обратите внимание, что при формировании строки на основе `log_loop_operation_fmt` нумерация итераций должна выполняться, начиная с единицы, а не с нуля.

В *IPC* из подготовительной работы следует внести следующие изменения: вызовы `read()` и `write()` должны быть неблокирующими (т.е. обмен сообщениями асинхронный).

В реализации запрещается использовать многопоточность: один процесс — один поток. Кроме того, нельзя использовать разделяемую память, примитивы синхронизации (семафоры и т.п.), функции `select()` и `poll()`. Логирование (`events.log`, `pipes.log` и терминал), как и в предыдущем задании.

При запуске программы с параметром командой строки «`--mutexl`» перед каждым вызовом `print()` процесс должен входить в критическую область, а после вызова выходить из нее, т.е. **запрещается выполнять несколько вызовов `print()` в пределах одной критической области**. При отсутствии параметра «`--mutexl`» программа должна выполняться без использования критической области, обратите внимание на разницу в выводе программы при использовании критической секции и без нее.

Вход в критическую область выполняется с помощью вызова функции `request_cs()`, выход — `release_cs()`. Обе функции необходимо реализовать самостоятельно, **используя алгоритм обедающих философов, описанный в методическом пособии и в лекционных презентациях. Алгоритм не должен быть централизованным.**

Процедура синхронизации процессов при запуске и завершении распределенной системы, как в предварительной работе.

Требования к реализации и среда выполнения

Реализацию необходимо выполнить на языке программирования Си с использованием предоставленных заголовочных файлов и библиотеки из архива [pa2345_starter_code.tar.gz](#).

Работа присылается в виде архива с именем `pa6.tar.gz`, содержащим каталог `pa6`. Все файлы с исходным кодом и заголовки должны находиться в корне этого каталога. Среда

выполнения — Linux (Ubuntu 16.04, clang-3.5). При автоматической проверке используется следующая команда: `clang -std=c99 -Wall -pedantic *.c -L. -lruntime`. При наличии варнингов работа не принимается. При успешном выполнении запущенные процессы не должны использовать `stderr`, код завершения программы должен быть равен 0.